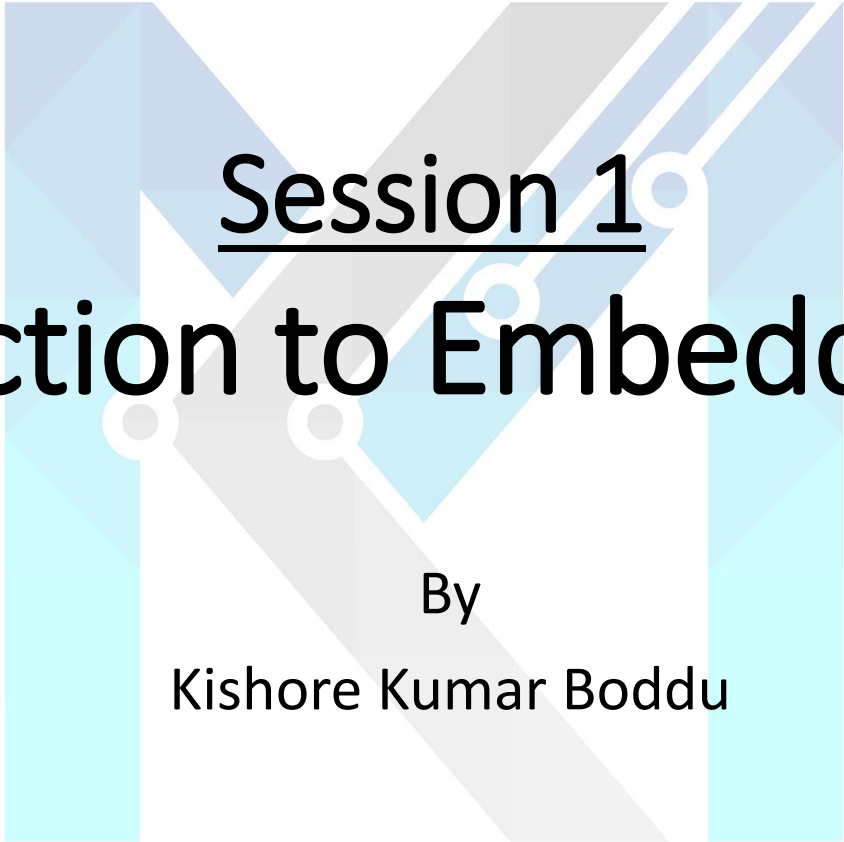




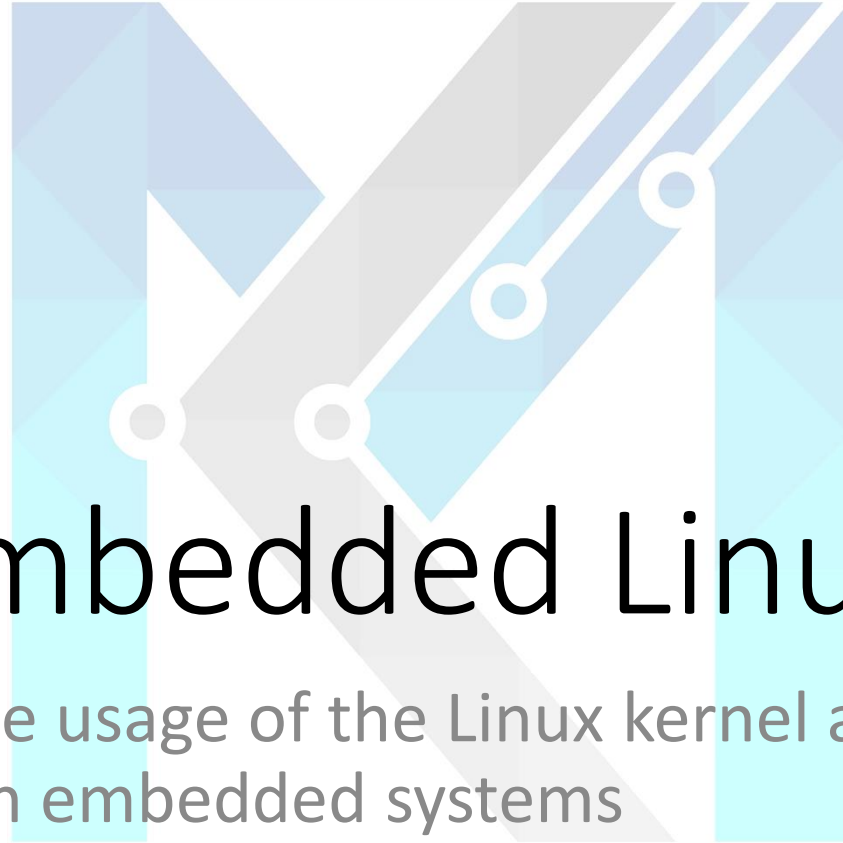
Session 1

Introduction to Embedded Linux

By

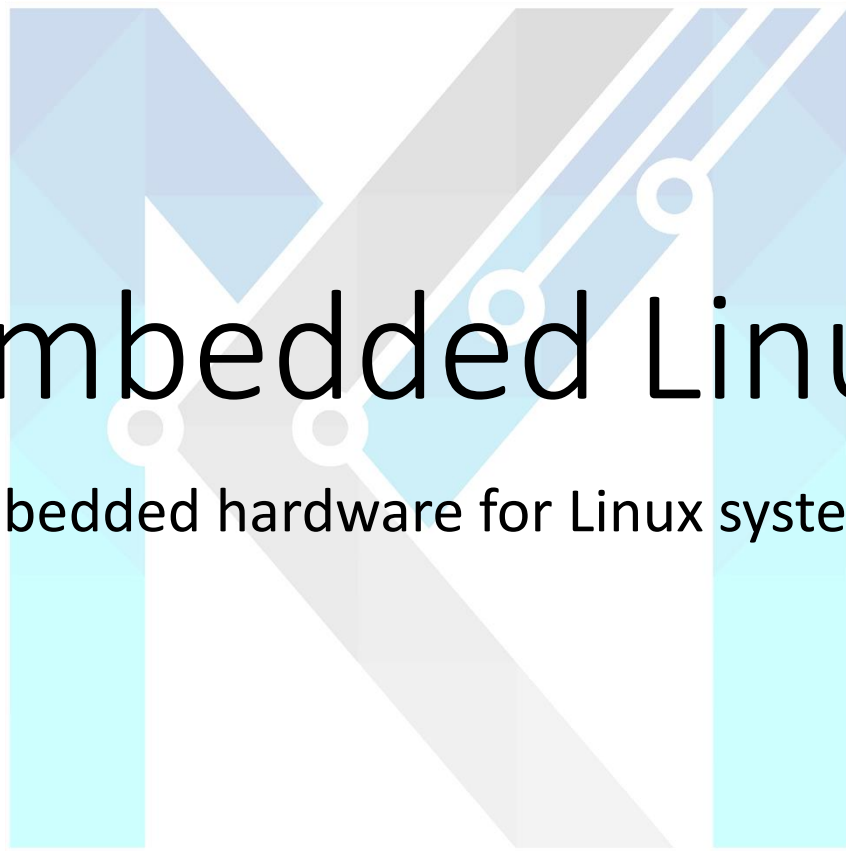
Kishore Kumar Boddu

- **Introduction to Embedded Linux**
 - What is Embedded Linux?
 - Embedded hardware
 - Embedded Software
- **Study AM335X SoC and BBB Board**
 - Read BBB SRM & TRM (AM335X & BBB Block Diagram)
 - AM335X Initialization/Boot sequence
 - AM335X Memory Mapping
- **Debian 10 Prebuilt Images porting on KM-BBB.**



What is Embedded Linux?

Embedded Linux is the usage of the Linux kernel and various open-source components in embedded systems



Embedded Linux

Embedded hardware for Linux systems?

Embedded Linux Hardware - Architecture

- The Linux kernel and most other architecture-dependent component support a wide range of 32 and 64 bits architectures
 - x86 and x86-64, as found on PC platforms, but also embedded systems (multimedia, industrial)
 - ARM, with hundreds of different SoC (multimedia, industrial)
 - PowerPC (mainly real-time, industrial applications)
 - MIPS (mainly networking applications)
 - SuperH (mainly set top box and multimedia applications)
 - Blackfin (DSP architecture)
 - Microblaze (soft-core for Xilinx FPGA)
 - Coldfire, SCore, Tile, Xtensa, Cris, FRV, AVR32, M32R

Embedded Linux Hardware - MMU

- Both MMU and no-MMU architectures are supported, even though no-MMU architectures have a few limitations.
- Linux is not designed for small microcontrollers.
- Besides the toolchain, the bootloader and the kernel, all other components are generally architecture-independent

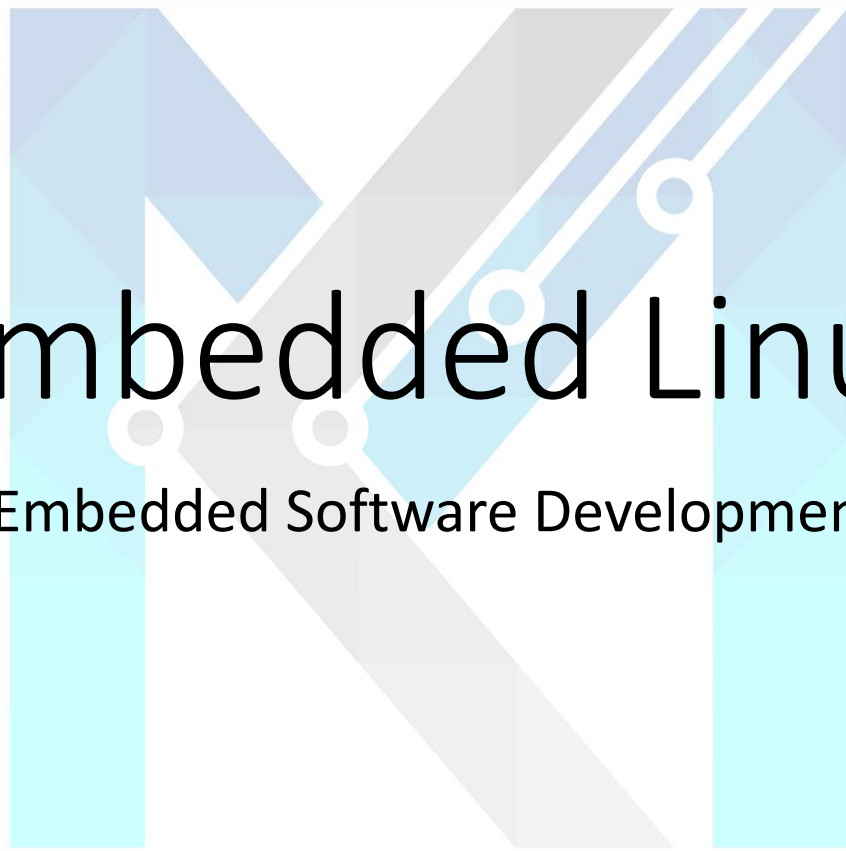
Embedded Linux Hardware – Communication Protocols

The Linux kernel has support for many common communication busses

- I2C
- SPI
- CAN
- 1-wire
- SDIO
- USB
- And also extensive networking support
 - Ethernet, Wi-Fi, Bluetooth, CAN, etc.
 - IPv4, IPv6, TCP, UDP, SCTP, DCCP, etc.
 - Firewalling, advanced routing, multicast.

Types of hardware platforms

- **Evaluation platforms** from the SoC vendor. Usually expensive, but many peripherals are built-in. Generally unsuitable for real products.
- **Component on Module**, a small board with only CPU/RAM/flash and a few other core components, with connectors to access all other peripherals. Can be used to build end products for small to medium quantities.
- **Community development platforms**, a new trend to make a particular SoC popular and easily available. Those are ready-to-use and low cost, but usually have less peripherals than evaluation platforms. To some extent, can also be used for real products.
- **Custom platform**. Schematics for evaluation boards or development platforms are more and more commonly freely available, making it easier to develop custom platforms.



Embedded Linux

Embedded Software Development

Software components

- **Cross-compilation toolchain**
 - Compiler that runs on the development machine, but generates code for the target
- **Bootloader**
 - Started by the hardware, responsible for basic initialization, loading and executing the kernel
- **Linux Kernel**
 - Contains the process and memory management, network stack, device drivers and provides services to userspace applications
- **C library**
 - The interface between the kernel and the userspace applications
- **Libraries and applications**
 - Third-party or in-house

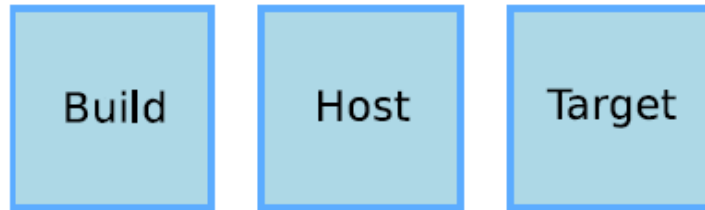
Cross-compiling toolchains

- The usual development tools available on a GNU/Linux workstation is a **native toolchain**
- This toolchain runs on your workstation and generates code for your workstation, usually x86
- For embedded system development, it is usually impossible or **not interesting to use a native toolchain**
- The target is too restricted in terms of storage and/or memory
- The target is very slow compared to your workstation
- You may not want to install all development tools on your target.
- Therefore, cross-compiling toolchains are generally used. They run on your workstation but generate code for your target.

Machines in build procedures

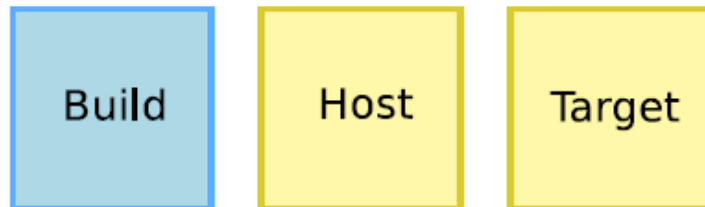
- Three machines must be distinguished when discussing toolchain creation
- The **build machine**, where the toolchain is built. (linaro community)
- The **host machine**, where the toolchain will be executed.
- The **target machine**, where the binaries created by the toolchain are executed.
- Four common build types are possible for toolchains

Machines in build procedures



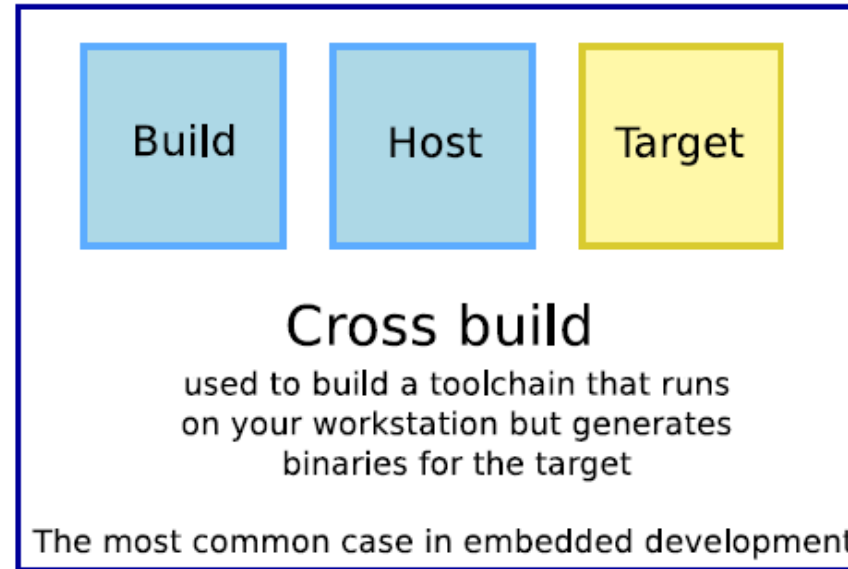
Native build

used to build the normal gcc
of a workstation



Cross-native build

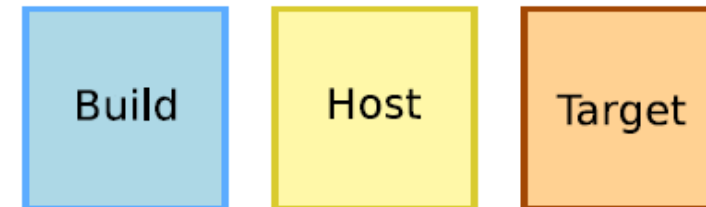
used to build a toolchain that runs on your
target and generates binaries for the target



Cross build

used to build a toolchain that runs
on your workstation but generates
binaries for the target

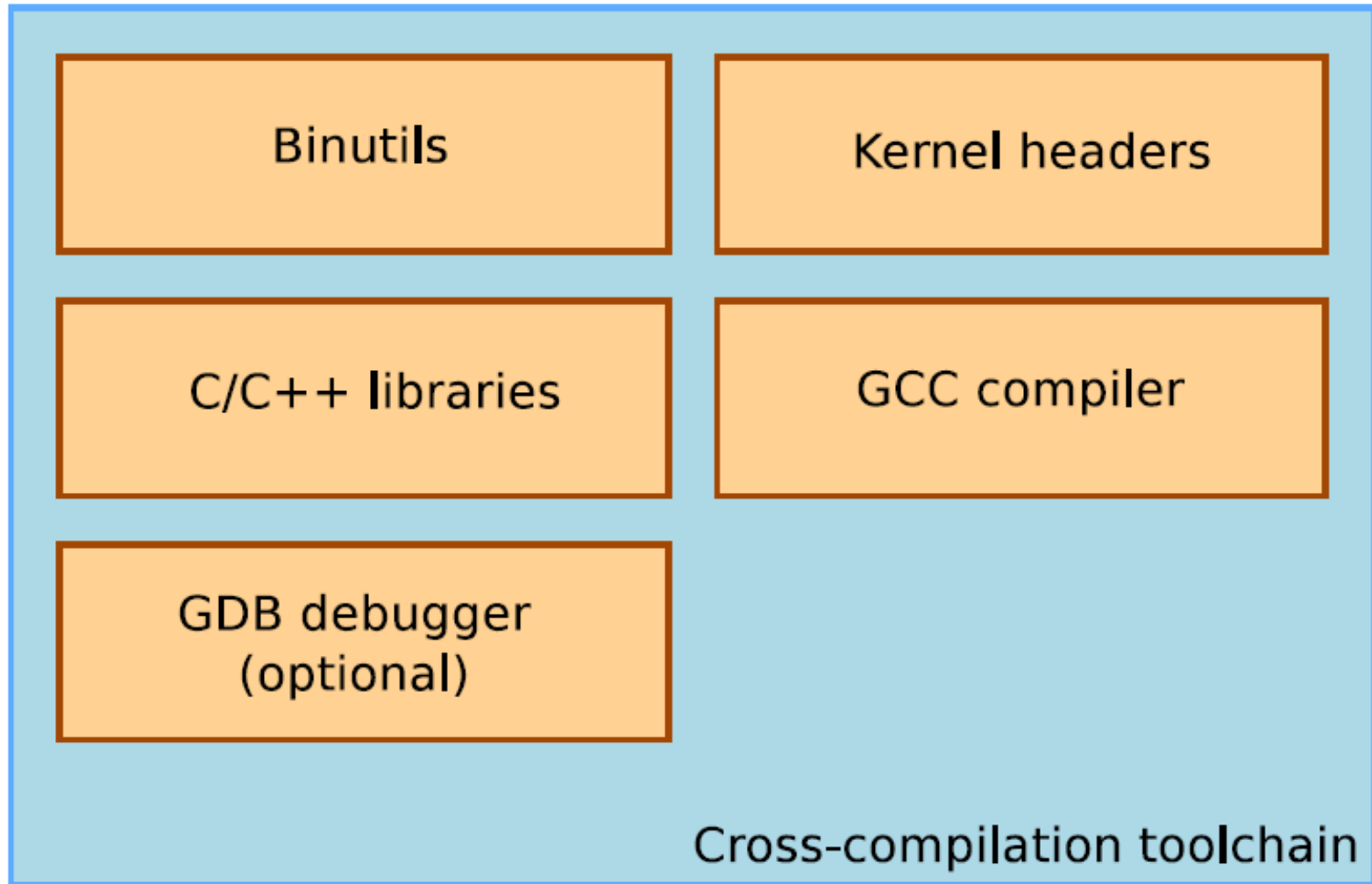
The most common case in embedded development



Canadian build

used to build on architecture A a
toolchain that runs on architecture B
and generates binaries for architecture C

Components



- Binutils is a set of tools to generate and manipulate binaries for a given CPU architecture
 - as, the assembler, that generates binary code from assembler source code
 - ld, the linker
 - ar, ranlib, to generate .a archives, used for libraries
 - objdump, readelf, size, nm, strings, to inspect binaries. Very useful analysis tools!
 - strip, to strip useless parts of binaries in order to reduce their size
- <http://www.gnu.org/software/binutils/>
- GPL license

Kernel headers (1)

- The C library and compiled programs needs to interact with the kernel
 - Available system calls and their numbers
 - Constant definitions
 - Data structures, etc.
- Therefore, compiling the C library requires kernel headers, and many applications also require them.
- Available in <linux/...> and <asm/...> and a few other directories corresponding to the ones visible in include/ in the kernel sources

Kernel headers (2)

- System call numbers, in `<asm/unistd.h>`

```
#define __NR_exit 1
```

```
#define __NR_fork 2
```

```
#define __NR_read 3
```

- Constant definitions, here in `<asm-generic/fcntl.h>`, included from `<asm/fcntl.h>`, included from `<linux/fcntl.h>`

```
#define O_RDWR 00000002
```

- Data structures, here in `<asm/stat.h>`

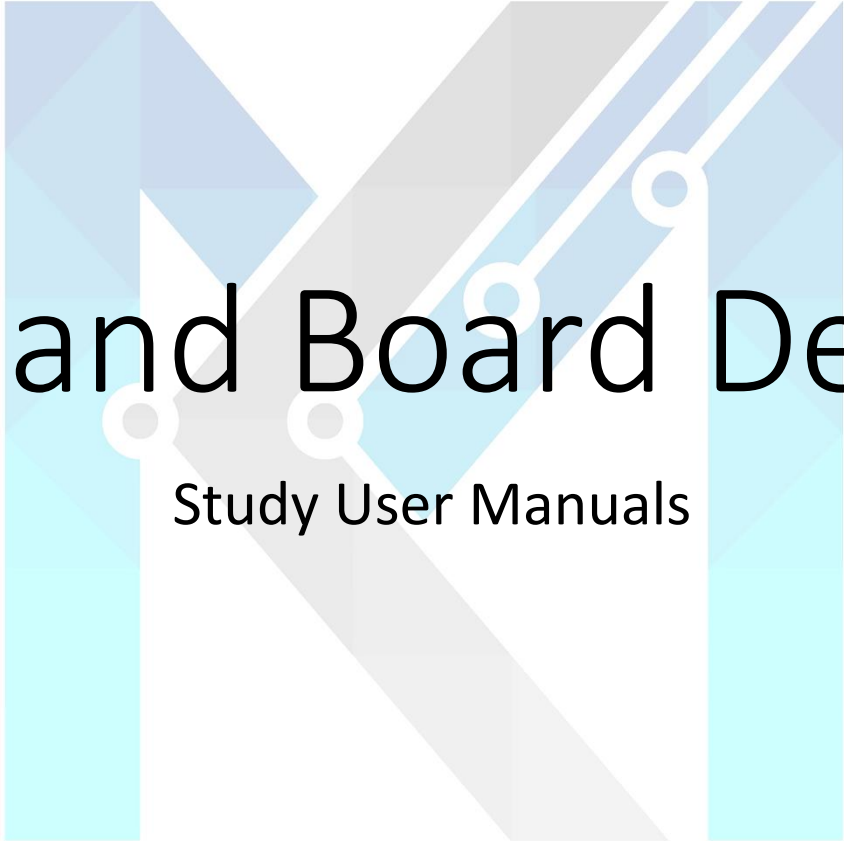
```
struct stat {  
    unsigned long st_dev;  
    unsigned long st_ino;  
    [...]  
};
```

Building a toolchain manually

- Building a cross-compiling toolchain by yourself is a difficult and painful task! Can take days or weeks!
 - Lots of details to learn: many components to build, complicated configuration.
 - Lots of decisions to make (such as C library version, ABI, floating point mechanisms, component versions).
 - Need kernel headers and C library sources
 - Need to be familiar with current gcc issues and patches on your platform
 - Useful to be familiar with building and configuring tools
 - See the Crosstool-NG docs/ directory for details on how toolchains are built.

Installing and using a pre-compiled toolchain

- Method 1:
 - Usually, it is simply a matter of extracting a tarball wherever you want.
 - Then, add the path to toolchain binaries in your PATH:
 - Export `PATH=/path/to/toolchain/bin/:$PATH`
- Method 2:
 - Install package
 - `$ sudo apt-get install gcc-arm-linux-gnueabi`



SoC and Board Details

Study User Manuals

AM3358 SoC Block Diagram

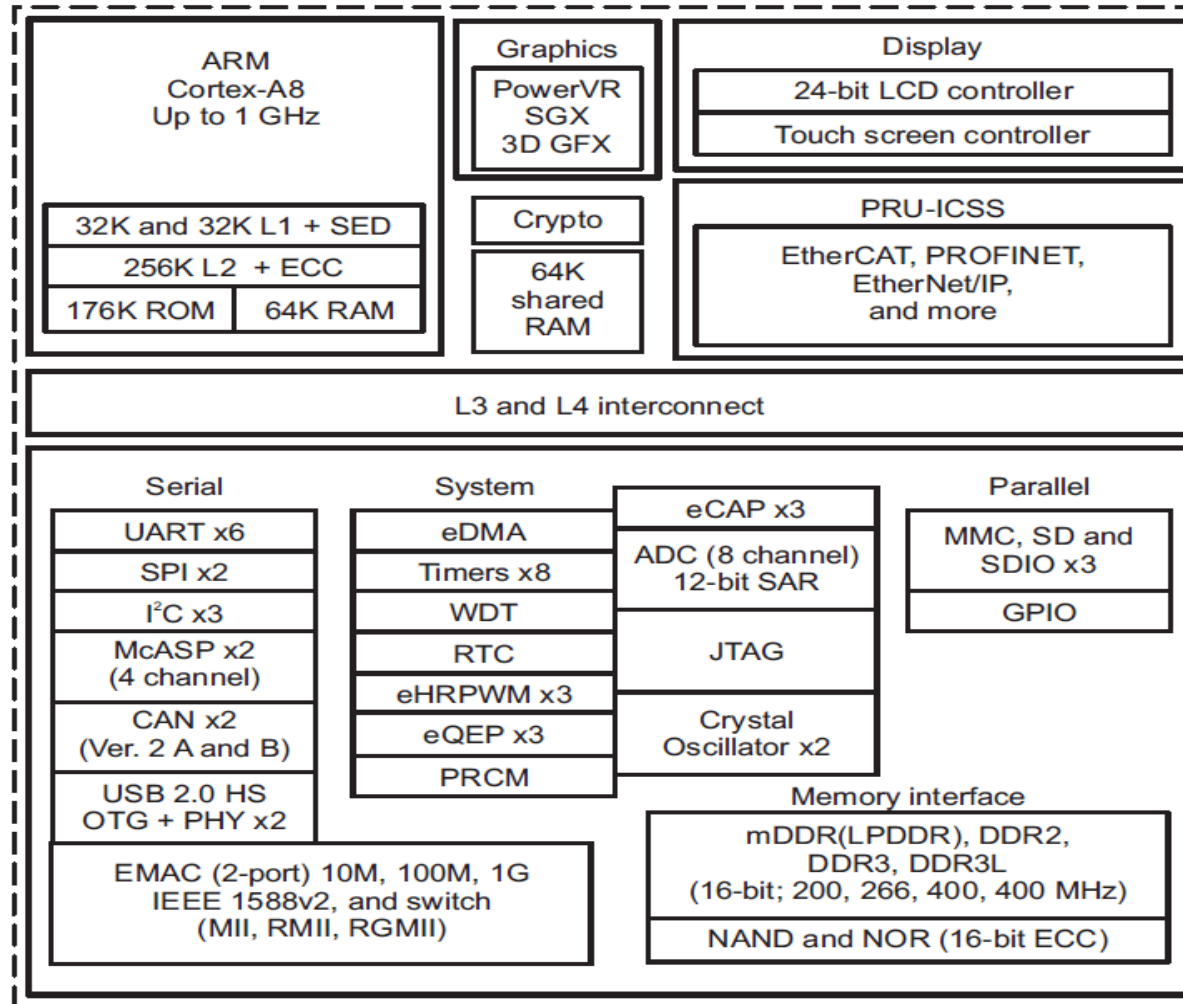
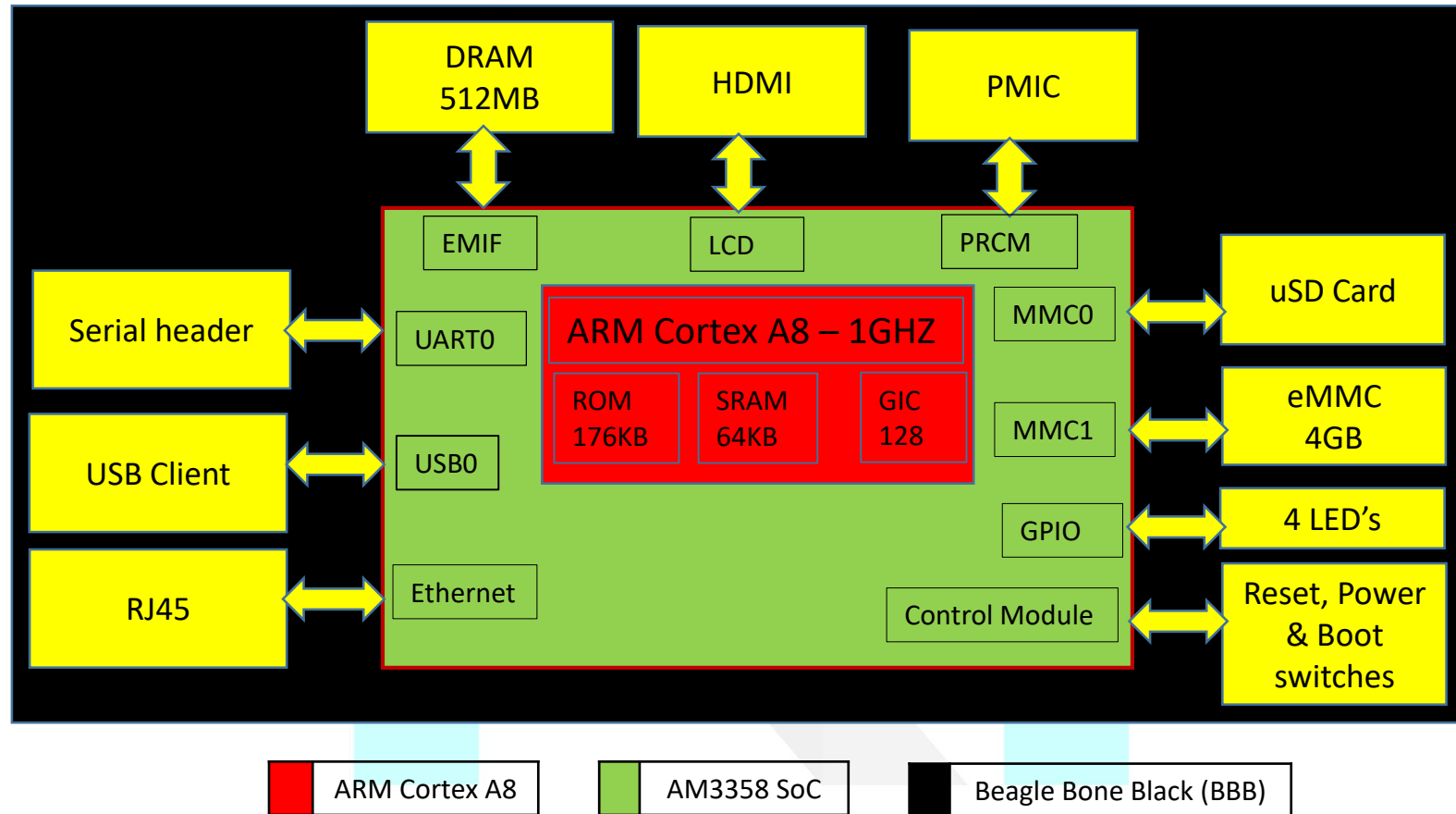
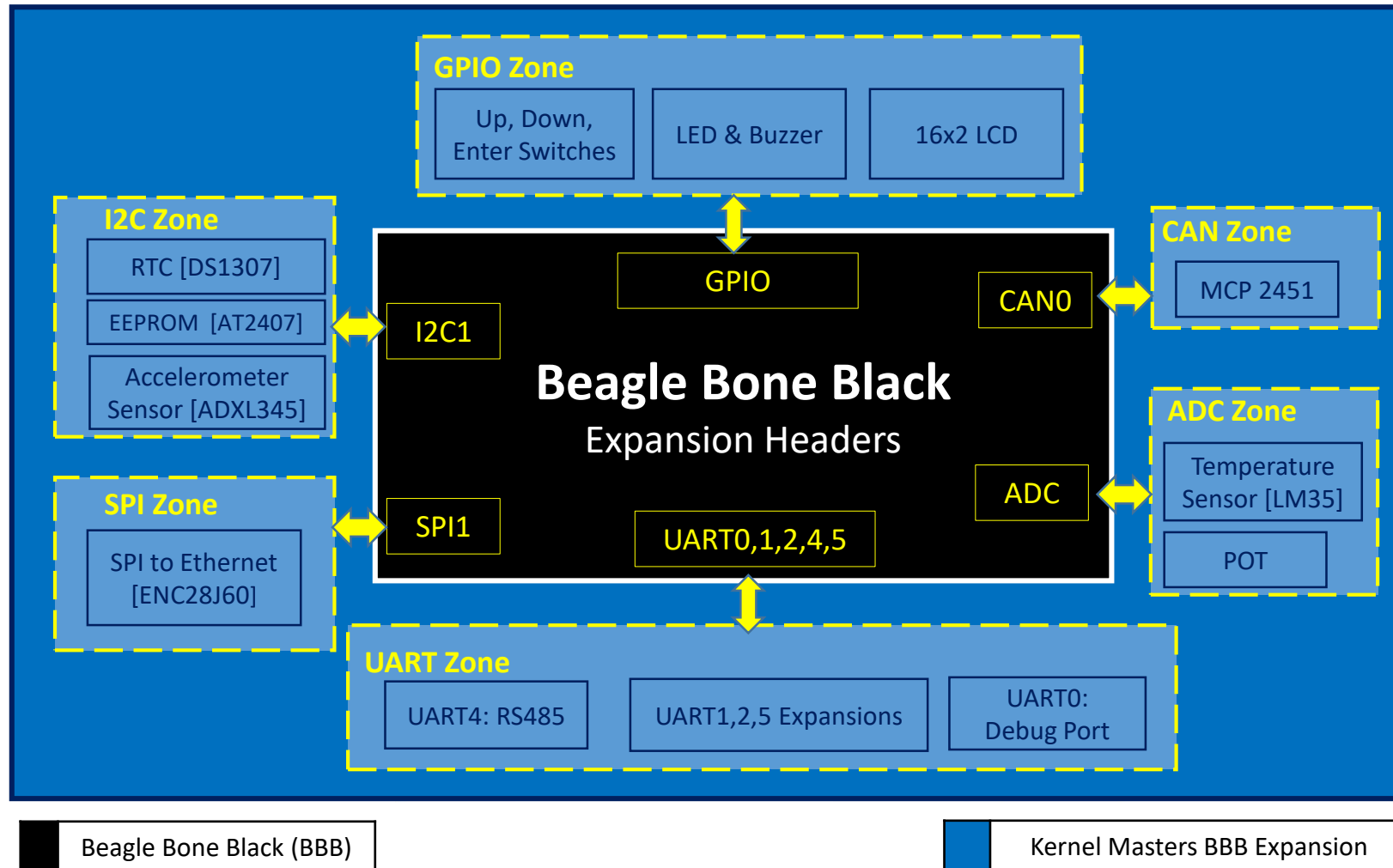


Figure 1-1. AM335x Functional Block Diagram

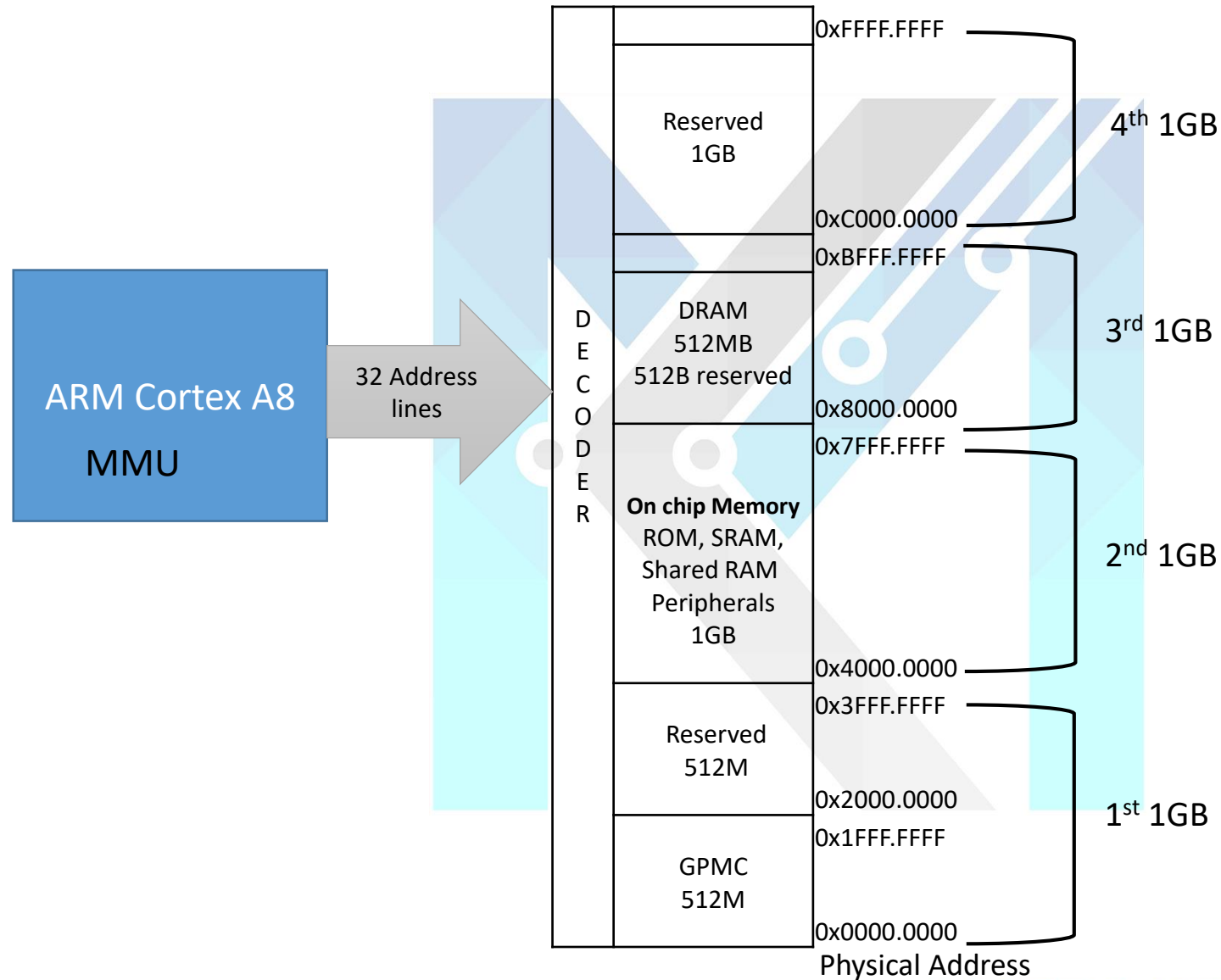
Beagle Bone Black Block Diagram

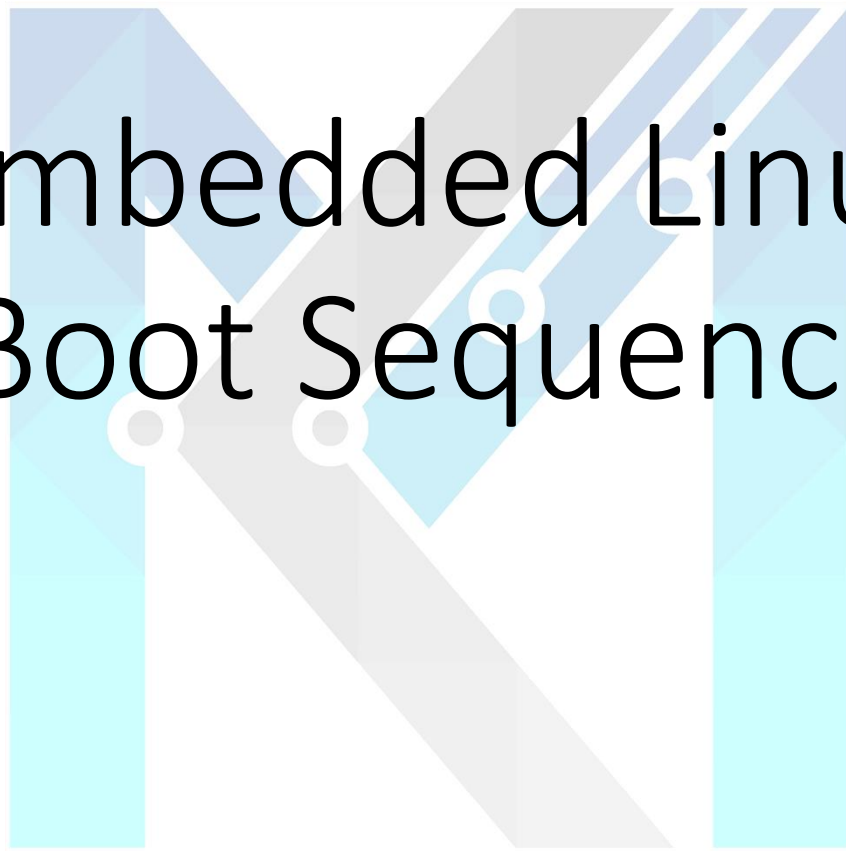


Kernel Masters Beagle Bone Black [KM-BBB] Expansion Board



AM335x Memory Mapping (Memory mapped I/O)





Embedded Linux Boot Sequence

Bootloaders

- The bootloader is a piece of code responsible for
 - Basic hardware initialization
 - Loading of an application binary, usually an operating system kernel, from ash storage, from the network, or from another type of non-volatile storage.
 - Possibly decompression of the application binary
 - Execution of the application
- Besides these basic functions, most bootloaders provide a shell with various commands implementing different operations.
- Loading of data from storage or network, memory inspection, hardware diagnostics and testing, etc.

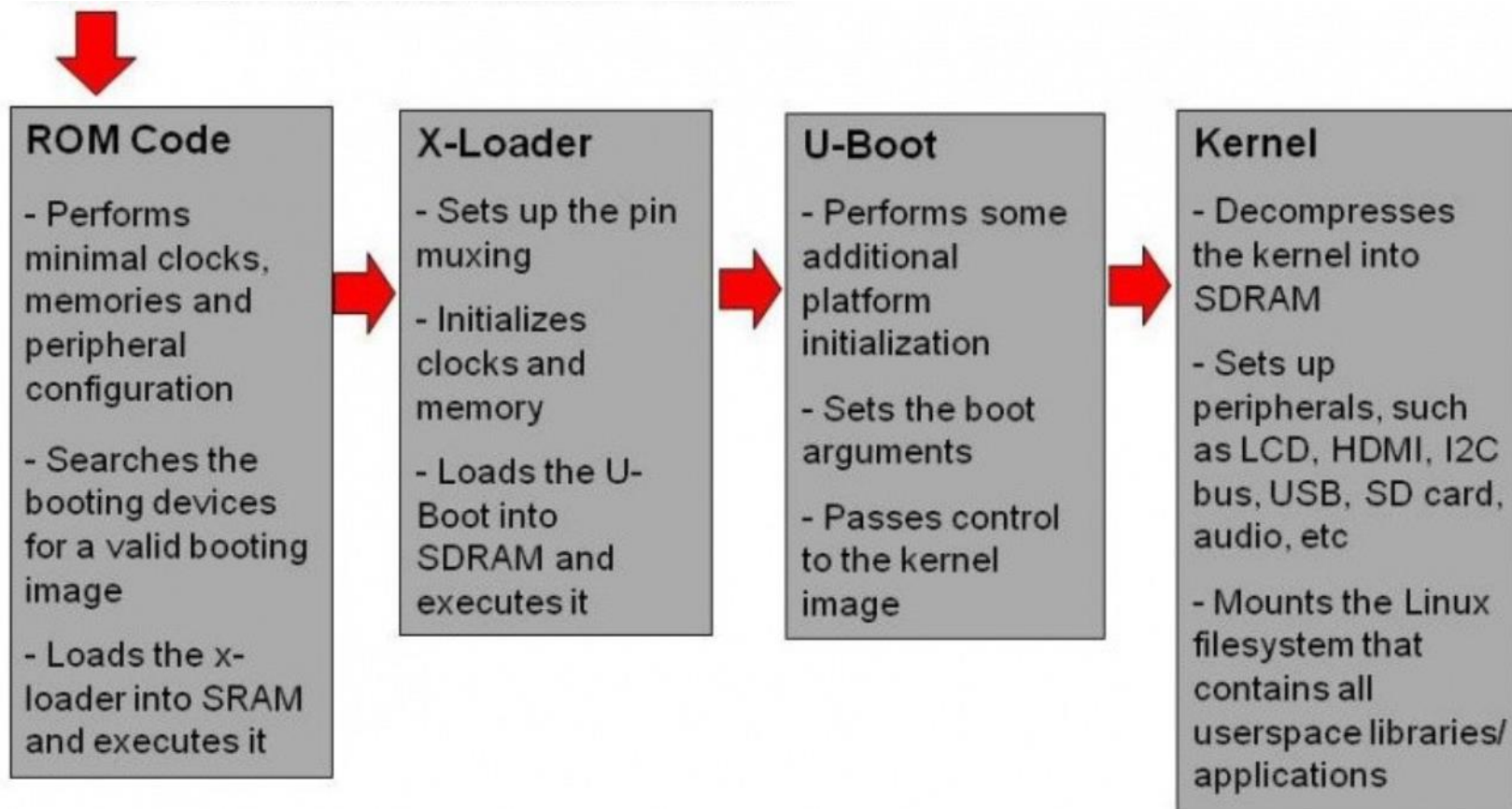
Bootloaders on x86

- The x86 processors are typically bundled on a board with a non-volatile memory containing a program, the BIOS.
- This program gets executed by the CPU after reset, and is responsible for **basic hardware initialization** and loading of a small piece of code from non-volatile storage.
- This piece of code is usually the first 512 bytes of a storage device
- This piece of code is usually a 1st stage bootloader, which will load the full bootloader itself.
- The bootloader can then offer all its features. It typically understands filesystem formats so that the kernel file can be loaded directly from a normal filesystem.

Embedded boot Sequence

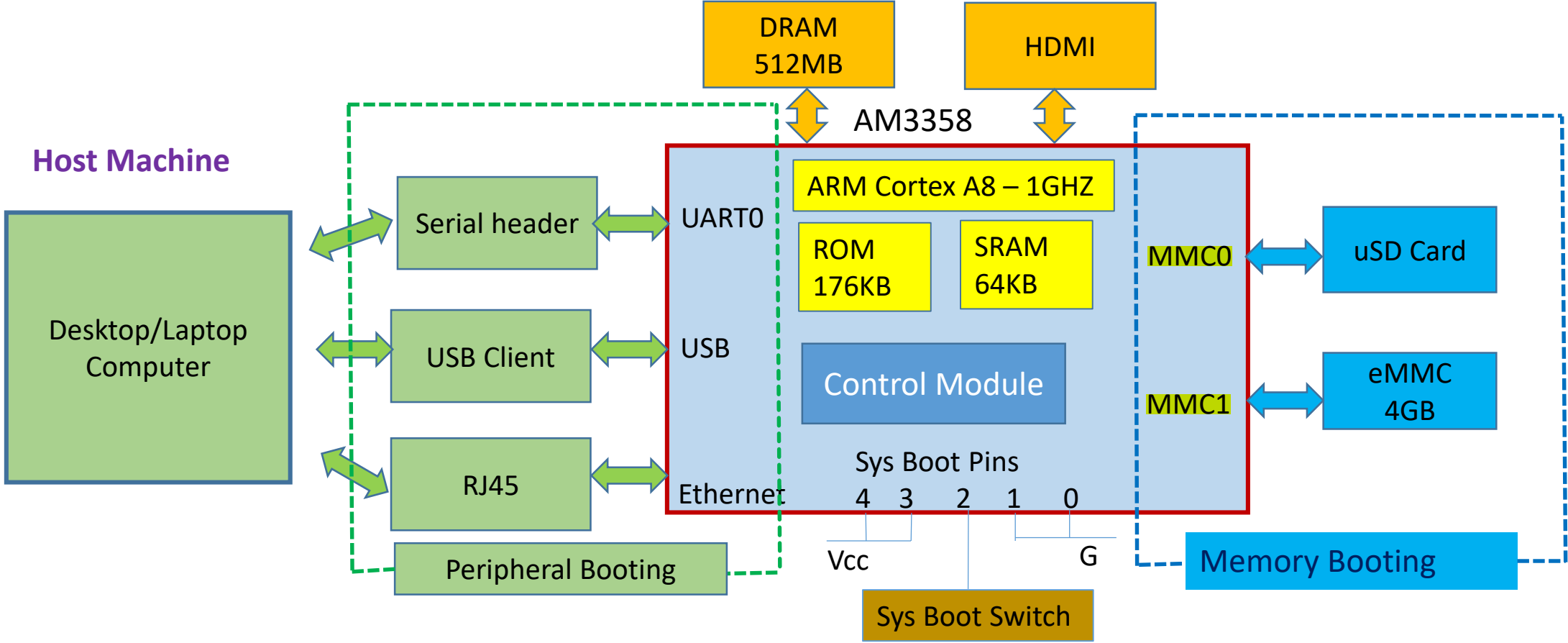
- **ROM Code:** tries to find a valid bootstrap image from various storage sources, and load it into SRAM or RAM (RAM can be initialized by ROM code through a configuration header). Size limited to <64 KB. **No user interaction possible.**
- **X-Loader or U-Boot:** runs from SRAM. **Initializes the DRAM**, the NAND or MMC controller, and loads the secondary bootloader into RAM and starts it. **No user interaction possible.** File called MLO.
- **U-Boot:** runs from RAM. Initializes some other hardware devices (network, USB, etc.). Loads the kernel image from storage or network to RAM and starts it. Shell with commands provided. File called u-boot.bin or u-boot.img.
- **Linux Kernel:** runs from RAM. Takes over the system completely (bootloaders no longer exist).

AM3358 boot Sequence



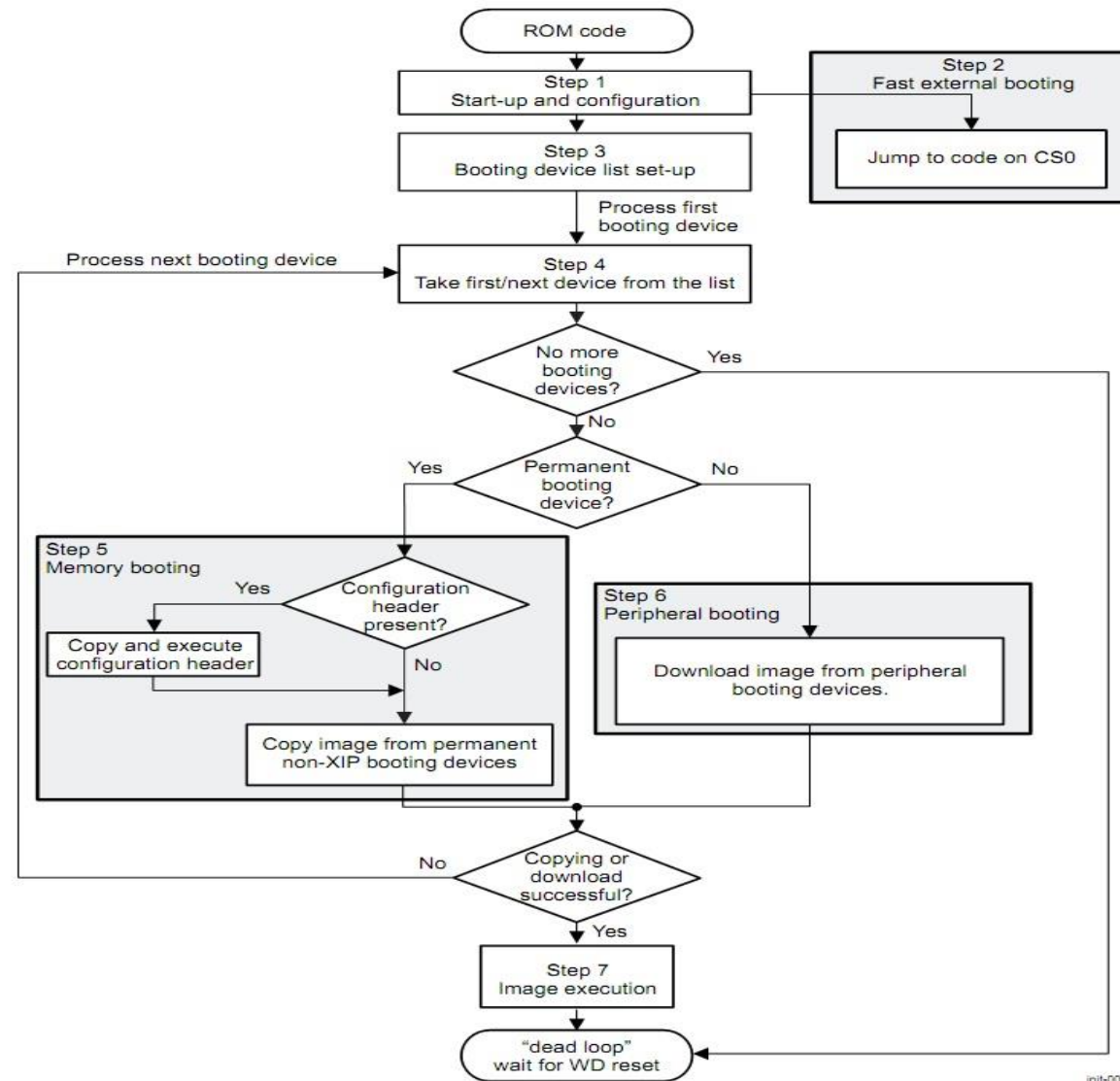
Embedded Linux Boot Sequence

Target Machine – Beagle Bone Black



Sysboot SW	Sysboot pins	1 st Order	2 nd Order	3 rd Order	4 th Order
OFF	11100	MMC1	MMC0	UART0	USB0
ON	11000	SPIO	MMC0	USB0	UART0

AM335x Boot sequence flowchart



init-007

Embedded Linux Boot Sequence



For more information about embedded Linux boot sequence click here:

<https://kernelmasters.org/blog/2020/06/30/embedded-linux-boot-sequence/>

Boot Loader Images

Image Name	Image Size	Image Header	Purpose
spl/u-boot-spl	2.2M	No	The binary of SPL ELF Image.
spl/u-boot-spl.bin	91264 Bytes	No	The second-stage bootloader (a stripped down version of u-boot that fits in SRAM).
spl/u-boot-spl.map	227K	No	contains the information for each symbol.
MLO	91784 Bytes	Yes, GP Header	spl/u-boot-spl.bin with a GP image header prepended to it.
U-boot.bin	474K	No	is the binary compiled U-Boot bootloader
u-boot.map	674K	No	contains the memory map for each symbol
U-boot.img	474K	Yes. GP header	contains u-boot.bin along with an additional header to be used by the boot ROM to determine how and where to load and execute U-Boot.

zImage vs uImage

Image Name	Purpose
Image	The generic Linux kernel binary image file.
zImage	a compressed version of the Linux kernel image that is self-extracting.
uImage	an image file that has a U-Boot wrapper (installed by the mkimage utility) that includes the OS type and loader information.
dtb	Device Tree Blob

⇒ `bootz <zImage_addr> - <dtb_addr>`

⇒ `bootm uImage`

zImage vs uImage

Recent versions of U-Boot can boot the zImage binary.

- Older versions require a special kernel image format: uImage
- uImage is generated from zImage using the mkimage tool. It is done automatically by the kernel make uImage target.
- On some ARM platforms, make uImage requires passing a LOADADDR environment variable, which indicates at which physical memory address the kernel will be executed.
- In addition to the kernel image, U-Boot can also pass a Device Tree Blob to the kernel.

The typical boot process is therefore:

1. Load zImage or uImage at address X in memory
2. Load <board>.dtb at address Y in memory
3. Start the kernel with bootz X - Y (zImage case), or bootm X - Y (uImage case)

The - in the middle indicates no initramfs

zImage vs uImage

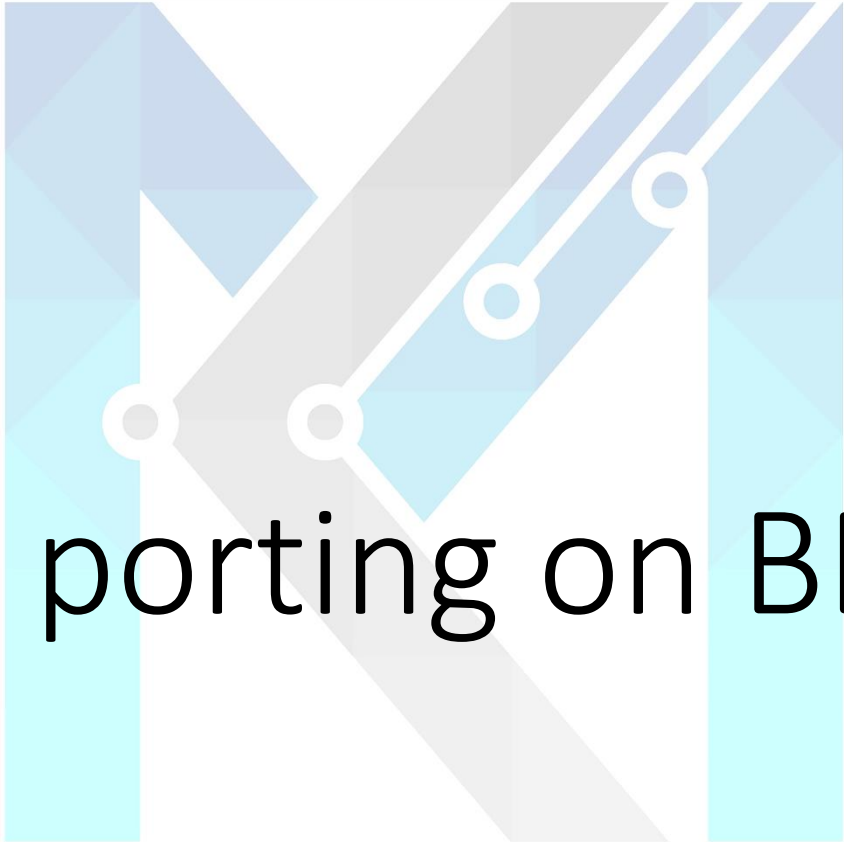
- A **uImage** includes a U-Boot header which contains the load address that the kernel should be loaded to. This seems nice at first, but then you realize that this makes this entire kernel specific to that load address, defeating the desire to have a kernel that can run "everywhere" (well, at least as broadly as possible).
- A **zImage** does not have this header and therefore the load address must be provided somewhere else. Typically, this is done with a bootz U-Boot command that takes the load address and the Device Tree File address as parameters:

```
bootz loadaddr - fdtaddr
```

U-Boot Image Format

U-Boot Image Formats:

- U-Boot operates on "image" files which can be basically anything, preceded by a special header; see the definitions in include/image.h for details;
- basically, the header defines the following image properties:
- Target Operating System (Provisions for OpenBSD, NetBSD, FreeBSD, 4.4BSD, Linux, SVR4, Esix, Solaris, Irix, SCO, Dell, NCR, LynxOS, pSOS, QNX, RTEMS, U-Boot, ARTOS, Unity OS, Integrity;
- Currently supported: Linux, NetBSD, VxWorks, QNX, RTEMS, ARTOS, Unity OS, Integrity).
- Target CPU Architecture (Provisions for Alpha, ARM, AVR32, BlackFin, IA64, M68K, Microblaze, MIPS, MIPS64, NIOS, NIOS2, Power Architecture®, IBM S390, SuperH, Sparc, Sparc 64 Bit, Intelx86;
- Currently supported: ARM, AVR32, BlackFin, M68K, Microblaze, MIPS, MIPS64, NIOS, NIOS2, Power Architecture®, SuperH, Sparc, Sparc 64 Bit, Intel x86).•
- Currently supported image compression types: none, gzip, bzip2, lzma, lzo, lz4.
- Load Address
- Entry Point
- Image Name
- Image Timestamp
- The header is marked by a special Magic Number, and both the header and the data portions of the image are secured against corruption by CRC32 checksums.



Debian 10 porting on BBB

Own built images

Debian 10 prebuilt images porting on KM-BBB

Debian 10 prebuilt images download from the below git repo:

\$ git clone [git@github.com:kernel-masters/debian10-km-bbb.git](https://github.com/kernel-masters/debian10-km-bbb.git)

And follow README.md file.

